Odoo Hackathon · Sales Operations Platform

DealFlow360

An intelligent, self-governing B2B quote-to-cash platform.

It explains the consequences of a deal, governs its approval, and carries the accepted terms through stock allocation and billing.

Discount governance | Live recommendations | Multi-warehouse split | Hybrid billing | Portal negotiation | Deal health

Team of 6 · Java 21 / Spring Boot 4 · React 19 + Vite · PostgreSQL · Supabase Auth

THE PROBLEM

B2B sales doesn't fit in a quote-to-invoice form

Six places where a real deal falls out of a simple sales tool

01
Discount leakage
Ceilings differ by customer tier AND product category. A rep can breach a category limit while the order-level number still looks fine.

02
Invisible approvals
Managers find out a deal is stuck after the momentum is already gone. Nobody owns the queue.

03
Inventory fiction
Stock is spread across warehouses. Quotes promise quantities that no single depot can actually ship.

04
Hybrid orders
One-time hardware and recurring subscriptions sit on the same order. Proration and credits get done by hand.

05
Email negotiation
Customers negotiate in threads. The agreed terms drift away from the document that gets approved.

06
Static PDFs
The quotation is a dead file, not a living, negotiable document both sides can act on.

Brief: “Most simple sales tools handle the basics well. Real B2B teams operate in messier conditions.”

PROBLEM STATEMENT

What the brief actually demanded

Seven responsibilities — all of them business logic, not screens

Responsibility	Required outcome	Where it lives
Multi-tier discount governance	Ceilings per customer tier and per category, automatic Manager → Finance routing, full audit history	policy · quotes · approvals
Live recommendations	Ranked upsell / cross-sell from co-purchase history, with real-time margin impact	recommendations
Multi-warehouse fulfillment	Stock-aware split across depots, manual override, backorder, consolidation on receipt	fulfillment
Hybrid billing	One-time and recurring lines on one order; schedules, proration, cancellation credits	billing
Portal negotiation	Genuinely separate restricted portal: line comments, counter-discounts, one-click confirm	portal
Deal monitoring	Stalled quotes, discount anomalies, promise slippage, nudges and escalations	health
Configuration & reporting	Working setup forms, report filters, PDF and XLS export	catalog · reporting

Non-negotiable constraints we designed against:
Core rules (approval routing, discount governance, warehouse splitting, billing proration) must be real application logic — not hardcoded or faked. The customer portal must be a real, separate, restricted view.

THE SOLUTION

One workspace that governs a deal from quote to cash

“A sales workspace that explains the consequences of a deal, governs its approval, and carries the accepted terms through stock allocation and billing.”

Build
Catalog + line editor with live pricing, margin and policy evaluation as you type.

Explain
Every evaluation returns structured reasons — why this approver, why this split, why this total.

Govern
Sequential Manager → Finance routing from a blended risk score; nothing inherits an approval.

Negotiate
Customer deal room with line comments, counters and exact-version acceptance.

Fulfil
Deterministic warehouse split with override, backorder and consolidation.

Bill
One-time and recurring on one order, with proration, credits and a real payment record.

UNIQUE SOLUTIONS

Six things we built beyond the checklist

These are the parts a CRUD demo cannot fake

- **Blended risk score, not a single threshold**
  Evaluates the worst line, value-weighted excess, total excess money, overall discount and margin — then routes to the highest level any of them demands. Splitting a line cannot lower the route, because a compliant line never offsets a breach (max(0, …) per line).

- **Three independent gates, and nothing inherits an approval**
  Approval, seller adoption and customer acceptance are separate gates in either order. Editing submitted terms supersedes the revision and retires its pending steps.

- **Finalisation is one database transaction**
  Order creation, stock reservation under row locks, and the first invoice commit together or not at all. Under in-stock-only terms a shortage rolls the whole thing back.

- **Explainability is a data structure, not a marketing word**
  Every engine emits structured reasons[] — “Finance is needed because the service discount exceeds its category ceiling by 8 points” — all of it rendered from real engine output.

- **Deterministic engines instead of ML theatre**
  Four pure-function engines (pricing, risk, allocation, proration) with no I/O. No trained model is required and none is claimed.

- **The portal is separate by construction**
  Portal responses use different DTO types with no field for cost, margin or policy thresholds. Out-of-scope reads return 404, not 403, so a probing client cannot map records.

SYSTEM DESIGN

How the pieces connect

One modular Spring service, one React SPA, one PostgreSQL database

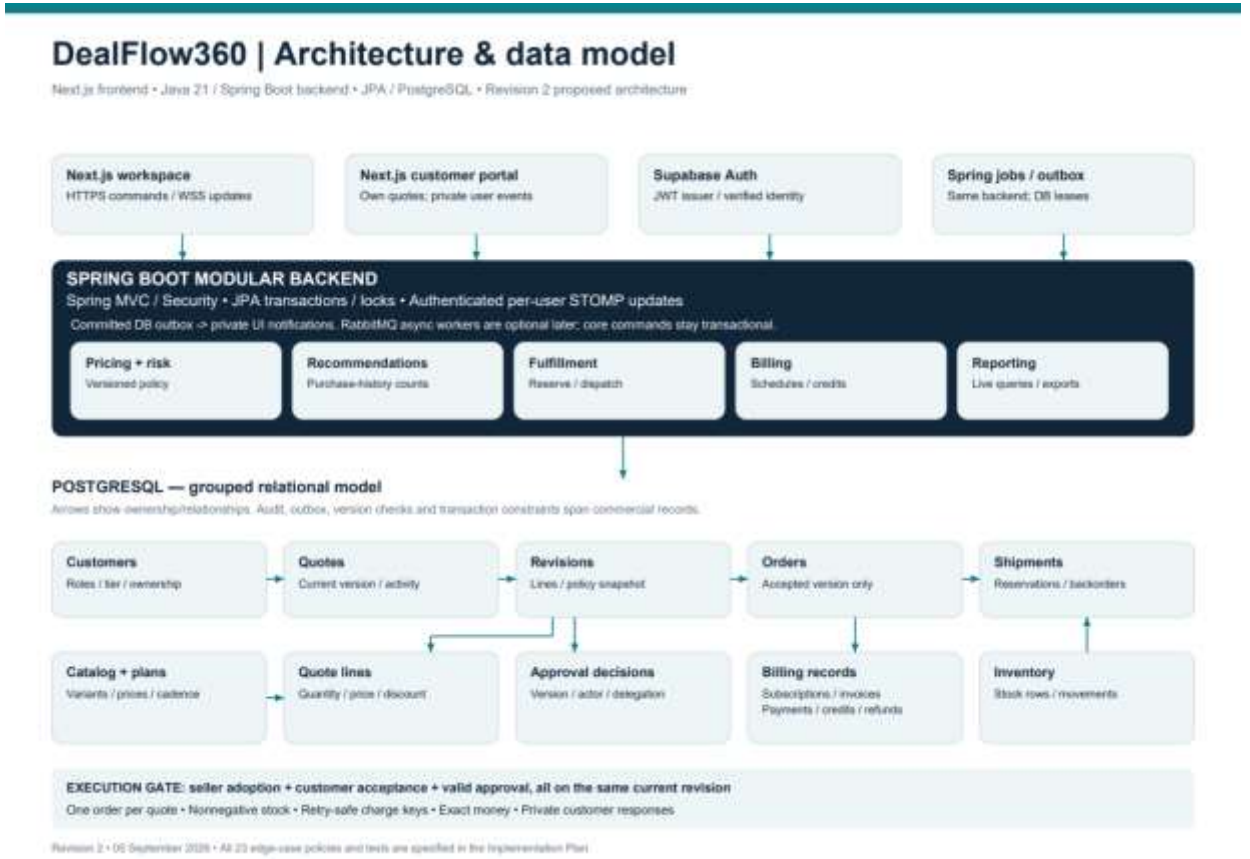


Diagram note: this architecture sheet predates the frontend swap — the delivered client is a Vite + React 19 SPA, not Next.js. The module boundaries, engines and data layer are as built.

SYSTEM DESIGN · BACKEND

One modular service, four pure engines

16 domain modules

auth · catalog · policy · quotes · approvals · portal · orders · fulfillment · billing · recommendations · health · reporting · outbox · notifications · jobs · assistant

220 Java files · 198 main + 22 test
7 Flyway migrations (V1–V7), Hibernate set to ddl-auto=validate — Flyway owns the schema.

Controllers validate and authorise. Services own transactions. The four engines are pure functions.

THE FOUR ENGINES

PricingEngine
Line + order discounts, tax, rounding

RiskEngine
Blended score → approval route

AllocationEngine
Warehouse split, shipping cost

ProrationCalculator
Calendar-based credits & adjustments

THE TRANSACTION THAT MATTERS

OrderFinalizationService.tryFinalize

Called from every path that can complete a gate. It locks the quotation, re-checks every gate, then creates the order, reserves stock and raises the first invoice — in one transaction.

Duplicate creation is blocked three ways: aggregate lock, explicit existence check, and a UNIQUE constraint on orders.quote_id.

Money: BIGINT minor units, computed in BigDecimal, rounded HALF_UP exactly once. Line residual is redistributed so lines always reconcile to the total.

SYSTEM DESIGN · INTEGRITY

The parts that break demos — and how we handled them

Failure mode	Our mechanism	Proven by
Two reps sell the last unit	Finalisation re-reads stock under (variant, warehouse)-ordered row locks; IN_STOCK_ONLY fails the whole transaction	ConcurrencyAndIdempotencyIT
Retry creates a second order or invoice	Aggregate lock + existence check + UNIQUE(orders.quote_id); recurring charges keyed by a global charge_key	ConcurrencyAndIdempotencyIT
Double-click on approve / pay / accept	Idempotency-Key header on every mutating endpoint — replay returns the original result, key reuse with a different body is refused	BillingLifecycleIT
Stale client overwrites a newer version	Optimistic row versions + exact-revision acceptance; stale writes rejected	FlowBNegotiationIT
Portal leaks cost, margin or policy	Separate portal DTOs with no such fields; out-of-scope reads return 404, not 403	FlowAEndToEndIT
A missed WebSocket frame loses an update	Transactional outbox → private per-user STOMP queue → REST refetch, plus a durable notifications inbox	WebSocketSecurityIT

SYSTEM DESIGN · REAL TIME

Live updates without lying to the user

01

Mutation commits

Every important mutation writes an outbox_events row inside the business transaction.

02

Dispatcher claims

A 1-second dispatcher claims rows with FOR UPDATE SKIP LOCKED and resolves recipients against current ownership.

03

Private push

A minimal frame (id, type, entity, version) goes to each user's private STOMP queue — never a broadcast.

04

Client refetches

The browser refetches through REST. Reconnect triggers an authorised refresh, because frames can be missed.

Security posture

Token in the CONNECT frame, not the URL. Subscriptions are per-user and authenticated; an anonymous CONNECT is refused and a foreign subscription closes the session. The WebSocket is never the source of truth — a committed outbox row is.

Jobs run in the background, not in the browser

A leased minute sweep raises due recurring charges (one transaction per subscription), runs deal-health rules and expires stale quotations. POST /internal/job-runs triggers the same code by hand — useful on stage.

TECH STACK

What we actually shipped it on

Layer	Choice	Why this and not the alternative
Frontend	React 19 + TypeScript, Vite 8, Tailwind v4, React Router 7	SPA with a dev proxy for /api and /ws. Role-based layouts for REP, MANAGER, FINANCE, ADMIN and CUSTOMER.
Backend	Java 21, Spring Boot 4.1, Spring MVC	Practical team experience, and a runtime that supports persistent WebSockets and scheduled jobs without extra infrastructure.
Persistence	PostgreSQL + Spring Data JPA / Hibernate 7, HikariCP, Flyway	Explicit locking, real constraints and versioned migrations. Flyway owns the schema; Hibernate only validates it.
Identity	Supabase Auth JWTs verified by Spring Security OAuth2 resource server	The token proves who signed in; the database decides what they may do. Roles are read from the DB on every request.
Real time	STOMP over native WebSocket (/ws) + @stomp/stompjs, transactional outbox	Authenticated per-user frames after commit, with REST refetch and a durable inbox as the safety net.
Money & time	BigDecimal + BIGINT minor units, java.time, BusinessClock	Exact rounding, calendar-correct proration, and an overridable clock for demos.
Exports	Apache PDFBox and Apache POI	Server-authoritative PDF, XLSX and legacy XLS — real bytes, not a print dialog.
AI	Sarvam chat completions (sarvam-105b), server-side only	Conversational explanation and review routing. No tools, no writes, and it never decides a price or an approval.
Verification	JUnit 5 + Testcontainers PostgreSQL, Playwright + axe	Rules are unit-tested; concurrency and auth are exercised against a real PostgreSQL.

PRODUCT SURFACE

25 routes, five role workspaces, one shared backend

Role	Landing route	What they can do there
Sales rep	/admin/sales/quotes	Pipeline list + kanban, and the Quote Builder: catalog, lines, live margin, recommendations, submit for approval
Manager	/admin/manager/approvals	Step-1 approval queue with the blended risk score and reasons; approve, reject or return for revision
Finance / Ops	/admin/finance/approvals	Step-2 commercial sign-off, warehouse allocation and backorders, billing and refunds
Admin	/admin/system/catalog	Master catalog, variants, price rules, discount policies, warehouses, plans, governance, jobs
Customer	/customer/quotes	A separate portal: their quotes only — comment per line, counter a discount, accept exact terms

Demo-day advantage: one-click persona switching

Eight seeded personas — admin, two reps, two managers, finance, and two customers (Alpha Traders / Bronze, Beta Systems / Gold). Switch role in one click and land on the right workspace, so a five-minute demo can show all five journeys without a single logout.

LIVE DEMO · FLOW A

The ordinary sale — no approval needed

Proves the happy path is genuinely automatic, not a shortcut

1	**Rep builds the quote**	Pick laptop + dock, apply a within-policy discount. Live margin updates on every keystroke.
2	**Accept a recommendation**	The panel ranks cross-sell from real confirmed-order co-purchases. Add it — the totals and margin move immediately.
3	**No approval triggered**	Risk engine finds nothing over ceiling, so the quote goes straight to share. No manager is disturbed.
4	**Customer accepts**	Exact-version acceptance in the portal. Gates are re-checked before anything is created.
5	**Stock reserved, invoice raised**	One transaction reserves across depots and raises the first invoice. Record a payment — status flips to paid.

LIVE DEMO · FLOW B

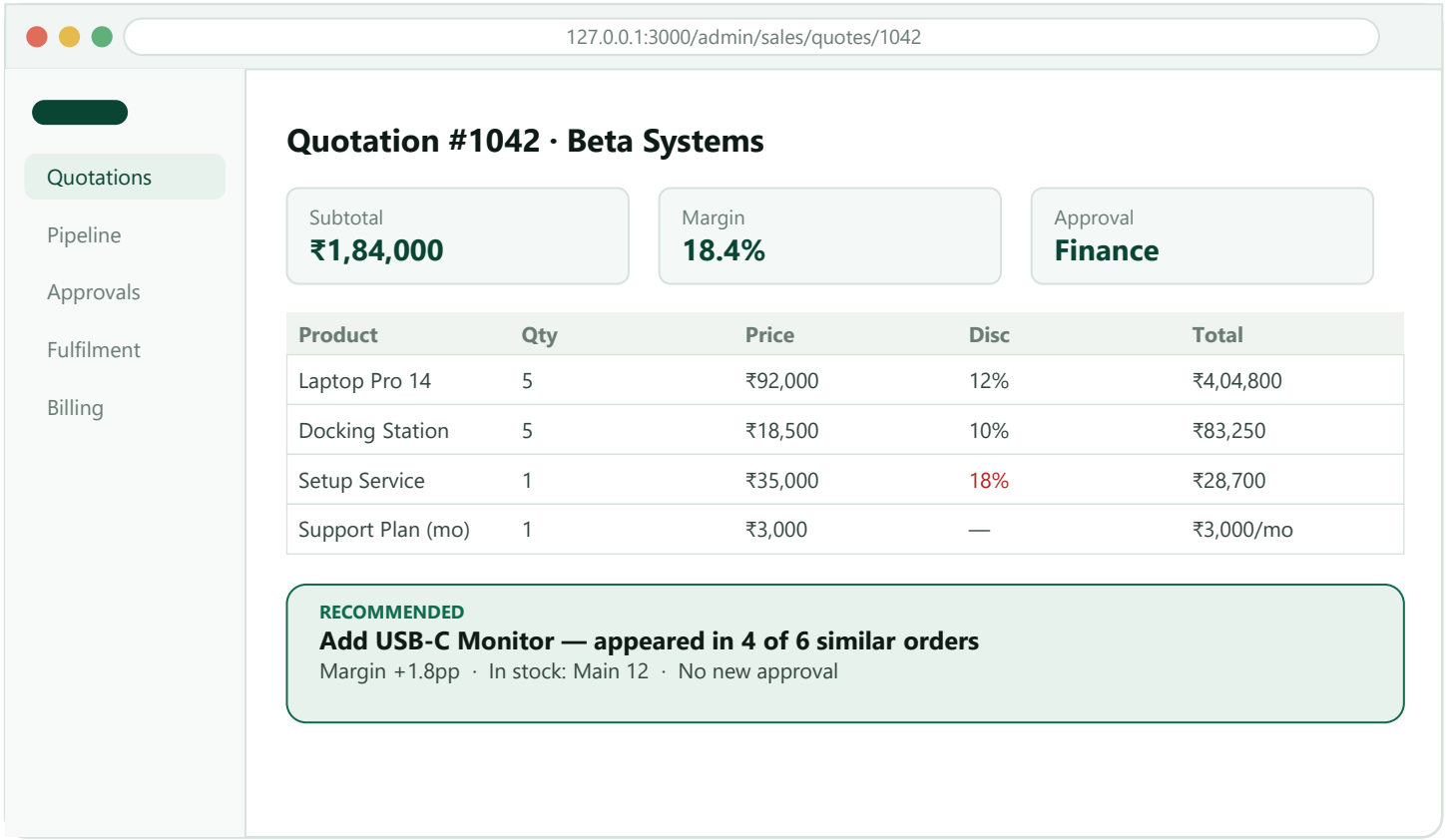
The messy deal — approval, negotiation, backorder

Seeded stock: Main 4 laptops / 10 docks, East 1 laptop

1	**Mixed quote breaches a category ceiling**	Hardware 12% (ceiling 15%) is fine; Setup Service 18% against a 10% ceiling is 8 points over — the whole quote is flagged.
2	**Manager, then Finance**	Sequential routing. Finance cannot be skipped and cannot act before the Manager step.
3	**Customer counters in the portal**	They propose a bigger discount on a line. That creates a candidate revision — it does not silently change the deal.
4	**Approvals are invalidated**	The counter re-enters the approval flow from step one. Nothing inherits the earlier sign-off.
5	**Split across two depots + backorder**	5 laptops against 4 in Main and 1 in East. Order 6 and it ships what exists, backorders the rest, then consolidates on receipt.
6	**Hybrid billing**	One-time lines invoice now; the subscription becomes a calendar-month schedule with a preserved anchor day. Change quantity mid-cycle → prorated adjustment.

SCREEN WALKTHROUGH · 1

Quote Builder — the rep's cockpit



POINT AT THIS

1. **Catalog + line editor**
   Hardware, services and subscriptions in one picker; quantities and per-line or order-level discounts.

2. **Live margin indicator**
   Recomputed by PricingEngine on every change — not on save.

3. **Required approval + reasons**
   Shows which role is needed and exactly why, before the rep submits.

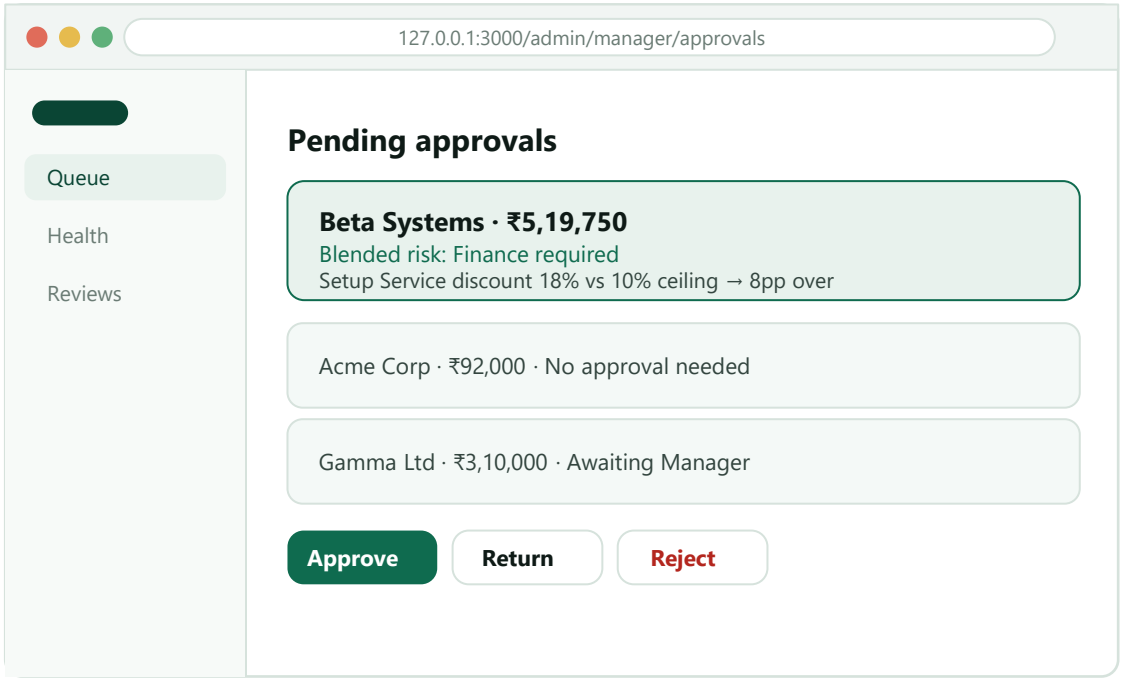
4. **Recommendation cards**
   Each offer previews customer price, deal net change, margin change, stock and required approval.

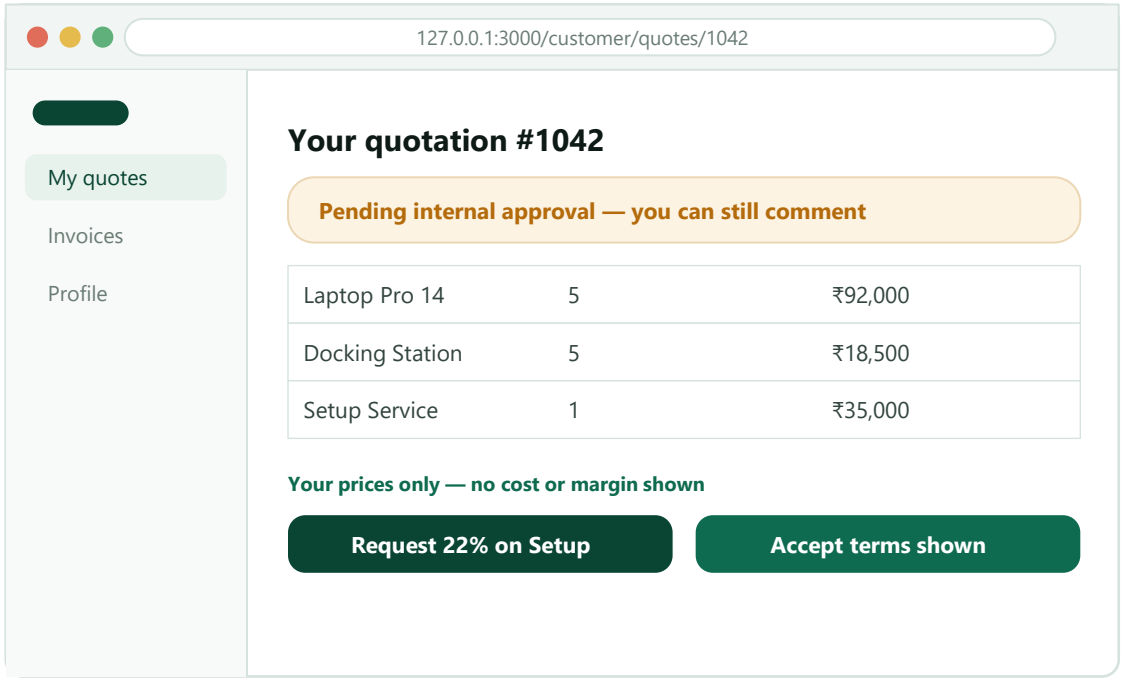
5. **Fulfilment availability**
   Preview only — reserving nothing until the gates pass.

SCREEN WALKTHROUGH · 2

Approvals and the customer deal room

Two screens that prove the governance is real





Manager approval queue

Blended risk score, the exact submitted revision, and the policy reasons. Approve, reject or return — every action attributed and time-stamped.

Customer deal room

Their quotations only — no cost, no margin, no thresholds. Comment per line, propose a counter, and accept the exact displayed version.

Say this out loud: an acceptance with outstanding approvals is conditional — it cannot release stock or create a billable order.

INTELLIGENCE

Recommendations and the assistant — with the brakes on

Recommendations

No trained model

- Cross-sell ranks on real confirmed-order co-purchases, promotion boost and contribution.
- Upsell proposes variants inside an explicitly configured compatibility group.
- Below five historical anchor orders it falls back to an explicitly labelled catalog result instead of inventing confidence.
- Applying requires the exact current revision and row version, so a stale offer is rejected.

Dismissals persist for the revision; unsaved edits block recommendation mutations.

Ask DealFlow assistant

Server-scoped, read-only, no tools

- The server decides identity and scope from the database; the browser cannot supply a role.
- A customer's context excludes internal costs, margins, policy ceilings and review notes.
- Plain-text answers. No executable tools. It cannot write to the database.
- It never creates a counteroffer, grants a discount, or accepts a quotation.
- Errors stay visible and retryable — there are no fake fallback answers.

The line we hold: AI explains the deal and routes questions. It does not price it, approve it, reserve stock or send anything on your behalf.

VERIFICATION

Evidence, not adjectives

112
JUnit test methods
across 20 test classes

7
Flyway migrations
V1–V7, validated

4
Engines unit-tested
pricing · risk · allocation · proration

5
Roles covered
rep, manager, finance, admin, customer

Concurrency
Two simultaneous acceptances produce one order. A last-unit race cannot oversell. Replayed idempotency keys return the original result.

Billing lifecycle
A prorated adjustment is created once on replay; backdating is refused; cancellation credits by segment and never refunds unpaid money.

Portal isolation
Another customer's quotation is inaccessible. A forged history role is rejected by the server, not the client.

Browser journeys
Playwright covers recommendation application, review routing, handoff, provider errors, keyboard focus and 320–1440px layouts, with axe a11y checks.

Counts above are measured from the source tree. Integration tests run against a real PostgreSQL via Testcontainers — a skipped Docker suite is not a pass.

WHAT WE'D BUILD NEXT

Honest roadmap

Deliberately cut to protect correctness — listed in priority order

01
Deal Lab
Generate and evaluate up to three valid alternatives for a stuck quote — approval-ready, margin-focused, delivery-focused — using the same engines. Structured reasons already exist.

02
Payment gateway
Payments and refunds are recorded receipts today. A real gateway is the obvious next integration.

03
Cadence changes
Quantity changes and cancellation are exposed; switching monthly → yearly is not.

04
Multi-currency & multi-company
Price records are currency-aware. The hackathon runs in INR with no FX conversion.

05
Distributed real time
The STOMP broker is in-process and single-instance. A second replica needs a broker relay first.

06
Email transport
Sharing a quotation returns a portal path today; it does not send mail.

Known gaps we will state before a judge finds them: no email transport, no payment gateway, Deal Lab not implemented, single backend instance. All documented in backend/README.md.

DealFlow360

Quote → Approval → Negotiation → Fulfilment → Billing → Reporting

Real business logic, real transactions, real audit trail.

Questions?